PIERNIK · BRANCH SF_STREAMINGCR_COMB · ON TOP OF DA40D857D

Moving star particles, CIC mapping, particle ids and SF heating: change log

Base commit da40d857d **Date** 11 October 2026 **Prepared for** Vinod V. Pisharody

The star particles of the `tallbox_sf` runs never moved. No par file had a `$PARTICLES` namelist, so the time integrator stayed at its default `'none'`. As a result, stars sat on their birth cells for the whole run, their mass never entered self-gravity, and their delayed SNe went off where they formed. Switching on `leapfrog2` exposed further bugs in the particle core, which are fixed here: no periodic wrapping, a crash for particles flagged outside the domain, wrong CIC mass deposition, ids written in single precision, and a segfault of nonmagnetic star-forming builds. Separately, the photoelectric heating in the `tallbox_sf` par files was 10× too high for the cooling table, which stopped star formation. Each record gives the code before and after and the reason. Line numbers refer to the files after the change.

16	**7**	**1.0000**	**+17%**
records: 7 source files, 6 par files, 2 analysis scripts	bugs fixed that froze particles, crashed runs or gave wrong physics	mapped particle mass / stellar mass (was −1.52 with CIC)	wall time per step with 14,700 moving particles (0.552 → 0.646 s, 20.8 pc, 6 ranks)

`BUG FIX` wrong results, crash or frozen state `PHYSICS` parameter change with physical effect `FEATURE` new behaviour or tool `PLUMBING` imports, interfaces, cosmetics

Contents

1–2	problem.par* (heating, particle integrator)	10–11	data_hdf5.F90, common_hdf5.F90 (ids)
3	particle_types.F90	12–13	star_formation.F90 (nonmagnetic builds)
4	particle_solvers.F90	14–15	analysis: plot_sfr.py, project.py
5–8	particle_utils.F90	16	build/run notes
9	particle_maps.F90	–	Validation, impact on earlier runs

Problem parameters: problems/tallbox_sf/problem.par, .A, .B, .C1, .C2, .D

1 Photoelectric heating `G1`

PHYSICS

`$THERMAL · problem.par.A line 106 (line 107 in the others)`

BEFORE

```
   G1            = 2.0e-25
```

AFTER

```
   G1            = 2.0e-26       ! Koyama & Inutsuka 2002 normalisation, matches cool_Kobayashi.txt
```

WHY `cool_Kobayashi.txt` is the Koyama–Inutsuka (2002) fit, whose heating normalisation is $\Gamma = 2\times10^{-26}$ erg s^{-1} per H (the table gives $\Lambda(100\ \mathrm{K}) = 1.1\times10^{-27}$, as in the fit). The value had been copied from `problems/tallbox` and was ten times larger. That shifted thermal equilibrium ($\Lambda(T) = G1/n$) so far that gas at the SF threshold stayed warm and failed the Jeans test:

ρ [$M_\odot$ pc^{-3}] (n [cm^{-3}])	T_{eq}, G1 = 2e-25	Jeans OK at 20.8 pc	T_{eq}, G1 = 2e-26	Jeans OK at 20.8 pc
0.35 (14)	2700 K	no	119 K	borderline
0.70 (28)	800 K	no	75 K	yes
1.0 (40)	460 K	no	62 K	yes
1.5 (61)	270 K	yes	51 K	yes

With the old value a 20.8 pc cell needed $n \gtrsim 60$ cm^{-3} (more at 10.4 and 7.8 pc), so the 20 pc case A formed no stars by 50 Myr. With the new value SF starts at about 12 Myr and reaches $\Sigma_{SFR} \approx 5\times10^{-3}$ $M_\odot$ yr^{-1} kpc^{-2} by 20–24 Myr, close to Kennicutt–Schmidt for $\Sigma_{gas} \approx 10$.

2 Particle integrator and mass deposition

BUG FIX

`new $PARTICLES namelist before $INTERACTIONS · problem.par.A lines 127–131`

BEFORE

```
(no $PARTICLES namelist: time_integrator = 'none', interpolation_scheme = 'ngp')
```

AFTER

```
 $PARTICLES
   time_integrator      = 'leapfrog2'   ! the default 'none' leaves star particles at their birth cells and out
of self-gravity
   interpolation_scheme = 'cic'         ! mass deposition onto the self-gravity source (acc_interp_method is
'cic' by default)
 /
```

WHY With `'none'`, `psolver` is null (`particle_solvers.F90:57`), so `update_particle_gravpot_and_acc` is never called. Particles were never moved: the 200–400 Myr displacement was exactly 0 while their stored |v| was about 15 pc Myr^{-1}. `nbdn` was mapped only once at t = 0 (`gravity.F90:509`), so stellar mass (1.4×10^6 $M_\odot$ by 400 Myr) was missing from self-gravity, and the delayed SNe of each star were injected in its birth cell. CIC matches the default CIC acceleration interpolation. It needed the fix in record 9.

src/particles/particle_types.F90

3 `is_outside` ignores periodic directions

BUG FIX

subroutine is_outside · lines 103–105

BEFORE

```
this%outside = any(dom%has_dir(:) .and. (this%pos(:) < dom%edge(:, LO) .or. this%pos(:) >= dom%edge(:, HI)))
```

AFTER

```
! Periodic directions do not count: physical particles are wrapped there (wrap_periodic) and periodic image
ghosts
! that lie beyond the edge are ordinary ghosts, not particles outside the domain.
this%outside = any(dom%has_dir(:) .and. .not. dom%periodic(:) .and. &
                   (this%pos(:) < dom%edge(:, LO) .or. this%pos(:) >= dom%edge(:, HI)))
```

WHY A particle that drifted across the periodic x/y edge was flagged `outside`. Its acceleration then came from `update_particle_acc_exp` → `moments2pot`, which dies unless gravity has isolated boundaries: `[multigridmultipole:moments2pot_xyz] Only fully isolated boundaries are implemented`. This happened about 0.2 Myr after `leapfrog2` was enabled. Only a non-periodic edge (z here) can now make a particle outside.

src/particles/particle_solvers.F90

4 Drift: periodic wrap, drop particles leaving a non-periodic edge

BUG FIX

leapfrog_2ord → subroutine drift · lines 423–474

BEFORE

```
use particle_utils, only: part_leave_cg, is_part_in_cg, detach_particle
...
if (pset%pdata%phy .and. pset%pdata%fin) then
   pset%pdata%pos = pset%pdata%pos + pset%pdata%vel * ddt
   call pset%pdata%is_outside()
   call is_part_in_cg(cgl%cg, pset%pdata%pos, .not.pset%pdata%outside, ...)
```

AFTER

```
use dataio_pub,     only: msg, warn
use multigridvars,  only: grav_bnd, bnd_isolated
use particle_utils, only: part_leave_cg, is_part_in_cg, detach_particle, wrap_periodic
...
logical :: drop_outside
drop_outside = (grav_bnd /= bnd_isolated)
...
if (pset%pdata%phy .and. pset%pdata%fin) then
   pset%pdata%pos = pset%pdata%pos + pset%pdata%vel * ddt
   call wrap_periodic(pset%pdata%pos)
   call pset%pdata%is_outside()
   if (pset%pdata%outside .and. drop_outside) then
      ! Beyond a non-periodic edge: the potential of particles outside the domain comes only from the multipole
      ! expansion, which needs isolated boundaries. Elsewhere the particle leaves the domain together with its
mass.
      write(msg, ...) "[particle_solvers:drift] particle ", pid, " of mass ", mass, " left the domain at ", pos
      call warn(msg)
      call detach_particle(cgl%cg, pset)
      cycle
   endif
   call is_part_in_cg(cgl%cg, pset%pdata%pos, .not.pset%pdata%outside, ...)
```

WHY The drift never wrapped positions, so particles drifted out of a periodic box. A particle leaving through the open z boundary (`outd`) would also hit the multipole `die` with these boundaries. It is now removed with a warning, like gas that flows out. With isolated gravity the old behaviour (keep it, multipole potential) is unchanged.

src/particles/particle_utils.F90

5 New `wrap_periodic` and `periodic_images`

FEATURE

public list lines 42–45 · new routines lines 631–699

BEFORE

```
(no such routines)
```

AFTER

```
integer, parameter :: max_images = 8    !< the particle itself and at most one periodic image per direction
(2**ndims)

subroutine wrap_periodic(pos)              ! pos(d) = edge_LO + modulo(pos(d) - edge_LO, L(d)) in periodic d
   ...
   if (pos(d) >= dom%edge(d, HI)) pos(d) = dom%edge(d, LO)   ! rounding in modulo

subroutine periodic_images(pos, nimg, img) ! img(:,1) = pos; one shifted copy per periodic direction
   margin = (npb + I_ONE) * dom%L_(d) / base%level%l%n_d(d)  ! wider than any ghost region
   if (pos(d) - dom%edge(d, LO) < margin) then ; shift = +L(d)
   else if (dom%edge(d, HI) - pos(d) <= margin) then ; shift = -L(d)
   else ; near_edge = .false. ; endif
   ! append img(:, i) + shift for every image so far (edges and corners)
```

WHY A particle near a periodic edge must also appear as a ghost just beyond the opposite edge, so that CIC/TSC deposition and the force interpolation see it there. Without images, the part of its mass that falls in guard cells was lost from the self-gravity source. Images are only generated within (npb+1) base cells of an edge, which costs nothing for particles away from the edges.

6 `attribute_to_proc` takes a position and an image flag

BUG FIX

function attribute_to_proc · lines 205–260

BEFORE

```
logical function attribute_to_proc(pset, j, se) result(to_send)
   ...
   if (pset%pdata%in .and. pset%pdata%fin) return
   ...
   if (particle_in_area(pset%pdata%pos, fbnd_npb(...))) then
      to_send = .true.
   else if (pset%pdata%outside) then
   ...
   if (to_send) then          ! NBODY_CHECK_PID
      if (j == proc .and. all(ll%dot%gse(j)%c(b)%se == se)) to_send = .false.
```

AFTER

```
!> \brief Should the particle pset, placed at pos (its own position or one of its periodic images), go to
process j?
logical function attribute_to_proc(pset, pos, is_image, j, se) result(to_send)
   real, dimension(ndims), intent(in) :: pos
   logical,                intent(in) :: is_image
   ...
   if (.not. is_image .and. pset%pdata%in .and. pset%pdata%fin) return
   ...
   if (particle_in_area(pos, fbnd_npb(...))) then
      to_send = .true.
   else if (pset%pdata%outside .and. .not. is_image) then
   ...
   if (to_send .and. .not. is_image) then          ! NBODY_CHECK_PID
```

WHY The routine decides where a particle gets ghost copies, so it must be able to test an image position. The early return and the same-cg suppression only make sense for the particle itself. An image may well land in the particle's own cg (when one cg spans the whole periodic width, as in the tall box).

7 `part_leave_cg` sends periodic images as ghosts

BUG FIX

subroutine part_leave_cg · counting lines 375–379, packing lines 404–416

BEFORE

```
if (attribute_to_proc(pset, j, cg%ijkse)) nsend(j) = nsend(j) + I_ONE
...
if (attribute_to_proc(pset, j, cg%ijkse)) then
   if (j == proc) then
      part_chcg(inc:inc+npf-1) = collect_single_part_fields(inc, pset%pdata)
   else
      part_send(ind:ind+npf-1) = collect_single_part_fields(ind, pset%pdata)
   endif
endif
```

AFTER

```
! Periodic images (im > 1) become ghosts beyond the periodic edges, so that mapping and interpolation see them
call periodic_images(pset%pdata%pos, nimg, img)
do im = 1, nimg
   if (attribute_to_proc(pset, img(:, im), im > 1, j, cg%ijkse)) nsend(j) = nsend(j) + I_ONE
enddo
...
call periodic_images(pset%pdata%pos, nimg, img)
do im = 1, nimg
   if (attribute_to_proc(pset, img(:, im), im > 1, j, cg%ijkse)) then
      pimg = pset%pdata
      pimg%pos = img(:, im)
      if (j == proc) then
         part_chcg(inc:inc+npf-1) = collect_single_part_fields(inc, pimg)
      else
         part_send(ind:ind+npf-1) = collect_single_part_fields(ind, pimg)
      endif
   endif
enddo
```

WHY This is where ghosts are created. Receivers add each copy through `add_part_in_proper_cg`, and `is_part_in_cg` marks an image beyond the edge as a ghost: `phy = .false.`, and `ext_bnd` is false in periodic directions, so `outdom_part_in_cg` cannot promote it. Images are therefore never integrated, written or counted as stars (`count_phy`), and they are removed at the next drift like any ghost.

8 Declarations in `part_leave_cg`

PLUMBING

lines 335–348

BEFORE

```
use constants,      only: I_ONE, PPP_PART, base_level_id
use particle_types, only: particle, npf
integer :: i, j, ind, inc
```

AFTER

```
use constants,      only: I_ONE, PPP_PART, base_level_id, ndims
use particle_types, only: particle, particle_data, npf
integer                            :: i, j, ind, inc, im, nimg
real, dimension(ndims, max_images) :: img
type(particle_data)                :: pimg
```

WHY Needed by record 7. `particle_data` contains no pointers, so `pimg = pset%pdata` is a plain copy.

src/particles/particle_maps.F90

9 CIC deposition used the wrong cell

BUG FIX

`subroutine map_cic · lines 144–146, 181`

BEFORE

```
use constants,      only: half, xdim, ydim, zdim, ndims, LO, HI, CENTER, PPP_PART
...
cn = nint((part%pos(cdim) - dom%edge(cdim, LO)) * cg%idl(cdim) + half) + 1
if (cg%coord(CENTER, cdim)%r(cn) > part%pos(cdim)) then
   ijkp(cdim, LO) = cn - 1
```

AFTER

```
use constants,      only: xdim, ydim, zdim, ndims, LO, HI, CENTER, PPP_PART
use particle_func,  only: ijk_of_particle
...
cn = ijk_of_particle(part%pos(cdim), dom%edge(cdim, LO), cg%idl(cdim))   ! cell containing the particle, as in
update_particle_acc_cic
if (cg%coord(CENTER, cdim)%r(cn) > part%pos(cdim)) then
   ijkp(cdim, LO) = cn - 1
```

WHY Grid indices are 0-based global indices, and the containing cell is `floor((x-edge)/dl)` (`ijk_of_particle`, also used by `map_ngp`, `map_tsc` and `update_particle_acc_cic`). The old formula is 1–2 cells too high: for (x−edge)/dl = 10.3 it picked cells 11–12 instead of 9–10. Then |Δx| > dl and the weights `1 − |Δx|/dl` were negative. In the test the integrated `nbdn` was −1.52× the stellar mass and uncorrelated with the gas (r = −0.02). It is now 0.999996×; the rest is star formation after the last mapping. The default NGP and TSC were not affected.

src/IO/hdf5/data_hdf5.F90 and common_hdf5.F90

10 Particle ids written as int32 in snapshots

BUG FIX

`data_hdf5.F90 · create_empty_cg_datasets_in_output · lines 1139, 1160–1164`

BEFORE

```
do i = lbound(pdsets,1), ubound(pdsets,1)
   call create_empty_cg_dataset(st_g_id, gdf_translate(pdsets(i)), [n_part], Z_avail, O_OUT)
enddo
```

AFTER

```
use hdf5,        only: H5T_STD_I32LE
...
do i = lbound(pdsets,1), ubound(pdsets,1)
   if (pdsets(i) == 'ppid') then  ! written from integers (particles_io:collect_intr); single precision would
merge ids > 2**24
      call create_empty_cg_dataset(st_g_id, gdf_translate(pdsets(i)), [n_part], Z_avail, O_OUT, H5T_STD_I32LE)
   else
      call create_empty_cg_dataset(st_g_id, gdf_translate(pdsets(i)), [n_part], Z_avail, O_OUT)
   endif
enddo
```

WHY Ids were collected and written as integers, but into a dataset created as `H5T_NATIVE_REAL` (float32, unless `h5_64bit`). Ids start at `proc·10^9/nproc` (1.67×10^8 on rank 1 of 6), far above 2^{24}, so HDF5's conversion merged neighbours. One snapshot had 356 distinct ids for 11,341 particles, which made particles impossible to track between outputs. Restart files are float64, which is exact, so the simulation itself was never affected.

11 Optional dataset type in `create_empty_cg_dataset`

PLUMBING

common_hdf5.F90 · lines 941, 955, 978

BEFORE

```
subroutine create_empty_cg_dataset(cg_g_id, name, ddims, Z_avail, otype)
```

AFTER

```
subroutine create_empty_cg_dataset(cg_g_id, name, ddims, Z_avail, otype, force_dtype)
   integer(HID_T), optional, intent(in) :: force_dtype !< HDF5 type overriding the one chosen by otype (e.g. for
integer data)
   ...
   if (present(force_dtype)) dtype = force_dtype
```

WHY Needed by record 10. All other callers are unchanged.

src/particles/star_formation.F90

12 `sf_fed` used `cg%b` in nonmagnetic builds

BUG FIX

subroutine sf_fed · lines 464–487

BEFORE

```
use constants,  only: xdim, ydim, zdim
use fluidtypes, only: component_fluid
use func,       only: emag
...
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) - emag(cg%b(xdim,i,j,k), cg%b(ydim,i,j,k), cg%b(zdim,i,j,k))
cg%u(pfl%ien,i,j,k) = frac1 * cg%u(pfl%ien,i,j,k)
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) + emag(cg%b(xdim,i,j,k), cg%b(ydim,i,j,k), cg%b(zdim,i,j,k))
```

AFTER

```
#ifdef MAGNETIC
use constants,  only: xdim, ydim, zdim
use func,       only: emag
#endif /* MAGNETIC */
use fluidtypes, only: component_fluid
...
#ifdef MAGNETIC
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) - emag(cg%b(xdim,i,j,k), cg%b(ydim,i,j,k), cg%b(zdim,i,j,k))
#endif /* MAGNETIC */
cg%u(pfl%ien,i,j,k) = frac1 * cg%u(pfl%ien,i,j,k)
#ifdef MAGNETIC
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) + emag(cg%b(xdim,i,j,k), cg%b(ydim,i,j,k), cg%b(zdim,i,j,k))
#endif /* MAGNETIC */
```

WHY `cg%b` is not allocated with `NONMAGNETIC`. Case A as built by `build_and_run.sh` (`-d H5PFC_EATS_1ST,NONMAGNETIC`) segfaulted at its first star-formation event: `func::sq_sum3 ← emag ← sf_fed ← SF`. Magnetic builds compile to the same code as before.

13 Same guard in `TIGRESS_injection`

BUG FIX

subroutine TIGRESS_injection · lines 1076–1079, 1107–1113 (inject_mass), 1169–1177 (equalize_region)

BEFORE

```
use func,                only: ekin, emag
...
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) - emag(cg%b(xdim,i,j,k), cg%b(ydim,i,j,k), cg%b(zdim,i,j,k))
...   (rescale)
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) + emag(cg%b(xdim,i,j,k), cg%b(ydim,i,j,k), cg%b(zdim,i,j,k))
```

AFTER

```
use func,                only: ekin
#ifdef MAGNETIC
use func,                only: emag
#endif /* MAGNETIC */
...
#ifdef MAGNETIC
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) - emag(...)
#endif /* MAGNETIC */
...   (rescale)
#ifdef MAGNETIC
cg%u(pfl%ien,i,j,k) = cg%u(pfl%ien,i,j,k) + emag(...)
#endif /* MAGNETIC */
```

WHY The two remaining `emag(cg%b...)` pairs (both in the `inject_mass` branch and in the `equalize_region` branch of the delayed SN injection) would crash in the same way in nonmagnetic builds.

Analysis: problems/tallbox_sf/analysis/

14 `plot_sfr.py`: no legend when nothing is plotted

PLUMBING

`main()` · lines 143–144

BEFORE

```
    ax.legend(fontsize="small")
```

AFTER

```
    if ax.get_legend_handles_labels()[0]:
        ax.legend(fontsize="small")
```

WHY Silences a matplotlib warning before the first complete SFR bin exists. The binning itself was checked on synthetic `.tsl` files with a restart overlap: it gives exactly the input SFR for both 1 and 10 Myr bins.

15 New `project.py`: multi-field projections

FEATURE

`new file, 276 lines`

USAGE

```
python ../problems/tallbox_sf/analysis/project.py RUNDIR [--axis x|y|z] [--fields dens temp pres vlos ...]
       [--zrange ZMIN ZMAX] [--clim dens=1e-2,1e2] [--every N] [--young 10] [--no-stars] [-o OUTDIR]
```

WHAT Reads the uniform-grid snapshots directly and writes one panel per field. Defaults: Σ (column), mass-weighted T, volume-weighted P/k_B, mass-weighted line-of-sight velocity. Any snapshot dataset can be added with `NAME:MODE` (sum, mean, mwmean, max, min, slice). `--axis z` gives face-on (xy) and `x` / `y` give edge-on views. Colour limits are fixed per field so movies are consistent. Star particles are overplotted in three classes: accreting sinks (`formation_time` = −10 until the first 100 $M_\odot$), sinks whose SN clock started within `--young` Myr, and older ones. `formation_time` is reset at every SN loadout, so it is not a birth time. About 1 s per snapshot at 20.8 pc.

16 Build and run notes

PLUMBING

No change to `build_and_run.sh`: it already builds A as `NONMAGNETIC` and B with MHD, and now works for A thanks to records 12–13. A plain `./setup tallbox_sf -p problem.par.A` builds an *MHD* binary. With `alpha = 1` it then starts with E_{mag}/E_{th} = 0.67, i.e. it is effectively case B. Particle output for analysis: `id` is now int32, and positions are wrapped into the box.

Validation

The tests were restarts of the 400 Myr local run (`tallbox_sf_A_h_0020.res`, 20.8 pc, 14,701 star particles, 6 ranks) for 1–2 Myr. They used an MHD build matching that run and a `NONMAGNETIC` case A build. Both gave the same particle results.

Check	Before	After
Particle displacement	`0 pc over 200 Myr`	`median 7.46 pc per 0.5 Myr, max 19 pc (|v| ≈ 15 pc/Myr)`
Periodic-edge crossings	— (crash at 400.2 Myr with leapfrog2)	`147–336 per 0.5 Myr, all particles inside the box`
Particles leaving through z	—	`0 in 2 Myr`
$\int$ nbdn dV / M_{stars}	`0 (never mapped) ; −1.52 with old CIC`	`0.999996`
Snapshot ids	`float32, 356 distinct / 11,341`	`int32, 14,712 distinct / 14,712`
NONMAGNETIC case A with SF	segfault in sf_fed	runs, SF and delayed SNe continue
Wall time per step	`0.552 s`	`0.646 s (+17%)`
SF onset, 20.8 pc (heating, record 1)	none by 50 Myr (reported)	`12 Myr; 5×10`$^{-3}$ `M`$_\odot$`/yr at 20–24 Myr`

Impact on earlier runs

- All `tallbox_sf` runs made before this commit had frozen stars: no motion, no stellar self-gravity, and SNe in the birth cells. Runs before 10 October also had the 10× heating. Production runs should restart from t = 0 with this commit.
- The local 0–400 Myr run in `runs/tallbox_sf` (run_id `A_h`) was built as MHD (see record 16) and has frozen stars. It is useful as a test, not as case A. It formed 1.44×10^6 $M_\odot$ in bursts at about 25, 130 and 250 Myr and lost about 62% of its gas through the z boundaries.
- Other problems that use NBODY with periodic boundaries (e.g. `sf`, `tallbox`) get the same particle fixes. Runs with CIC deposition anywhere in this tree were depositing wrong masses.

> Not changed: `pid_gen` is restored per process from the last cg it reads. If a restart gives cgs to different ranks, new ids could repeat old ones. Restarts with the same number of ranks are fine.